The printer of the denotation: the listing of statement P5

presk 0.13.0-dev, plan 0.13.0 step 4 (D137)

17 October 2026

This listing is what `presk dump --format latex` prints, unedited, in the notation of the language reference's chapter 3 and its section 3.10 (presk-specs !85, draft v0.18). Statement P5 of plan 0.13.0 asks the reference's author to check it: that the typed program's equations are chapter 3's, on the tutorial and the course GPT, and that the adjoints of `grad` are written as the chapter's rules write them.

1 The tutorial: the tower and its loss

Section 3.10 of the reference read on `tutorial/07_train.psk` and its imports: a linear layer, RMSNorm, a residual block, a stack of three blocks, the loss.

`01_linear.psk`

Module `Linear`$\{In, Out\}$. Its parameters and modules: $w : \mathbb{R}[In, Out]$, $b : \mathbb{R}[Out]$.
`Linear.apply`, of $x : \mathbb{R}[\ldots, In]$:

$$[\![\texttt{Linear.apply}]\!](\theta, x) = (\texttt{dot}\ x, w) + b$$

`02_norm.psk`

Module `RMSNorm`$\{D\}$. Its parameters and modules: $\gamma : \mathbb{R}[D]$.
`RMSNorm.apply`, of $x : \mathbb{R}[\ldots, \texttt{model}\colon D]$, $eps : \texttt{f64} = 10^{-6}$:

$$ms = \texttt{mean axis model}\ x^2$$
$$[\![\texttt{RMSNorm.apply}]\!](\theta, x, eps) = \frac{x}{\sqrt{ms + eps}} \times \gamma$$

`06_tower.psk`

Constants $D = 4$, $L = 3$.

Module `Block`. Its parameters and modules: $norm$: `norm.RMSNorm`$\{D\}$, $proj$: `lin.Linear`$\{D, D\}$.
`Block.apply`, of $x : \mathbb{R}[\ldots, \texttt{model}\colon D]$:

$$[\![\texttt{Block.apply}]\!](\theta, x) = x + [\![\texttt{proj.apply}]\!](\texttt{gelu}\ [\![\texttt{norm.apply}]\!](x))$$

Module `Tower`. Its parameters and modules: $blocks$: `stack`$\{L\}$(`Block`), $final$: `norm.RMSNorm`$\{D\}$.
`Tower.apply`, of $x : \mathbb{R}[\ldots, \texttt{model}\colon D]$:

$$y_0 = x$$
$$y_{\ell+1} = [\![\texttt{Block.apply}]\!](\theta.blocks[\ell], y_\ell) \qquad 0 \le \ell < L$$
$$[\![\texttt{Tower.apply}]\!](\theta, x) = [\![\texttt{final.apply}]\!](y_L)$$

`07_train.psk`

Dimensions B.

Function `loss`. Of θ : `params t.Tower`, $batch$: ($\mathbb{R}$[`batch`: B, `model`: 4], $\mathbb{R}$[`batch`: B, `model`: 4]):

$$(x, y) = batch$$
$$[\![\texttt{loss}]\!](\theta, batch) = \texttt{mean}\ ([\![\texttt{t.Tower.apply}]\!](\theta, x) - y)^2$$

2 The tutorial: attention

`tutorial/03_attention.psk`: an `ein` as its sum at an index named by its axes, the second operand relabeled `seq2` (definition 2.39).

`03_attention.psk`

Constants $D = 8$, $H = 2$, $HD = \frac{D}{H} = 4$; dimensions B, $T \leq 1024$; types $\texttt{Q} = \mathbb{R}[\texttt{batch}: B, \texttt{heads}: H, \texttt{seq}: T, \texttt{hd}: HD]$; asserted $D \bmod H = 0$.

Function `scores`. Of q : `Q`, k : `Q`:

$$[\![\texttt{scores}]\!](q, k)[\texttt{batch}, \texttt{heads}, \texttt{seq}, \texttt{seq2}] = \frac{\sum_{\texttt{hd}} q[\texttt{batch}, \texttt{heads}, \texttt{seq}, \texttt{hd}]\, k[\texttt{batch}, \texttt{heads}, \texttt{seq2}, \texttt{hd}]}{\sqrt{HD}}$$

Function `attend`. Of q : `Q`, k : `Q`, v : `Q`:

$$s = [\![\texttt{scores}]\!](q, k)$$
$$a = \texttt{softmax axis seq2}\ s, (\texttt{causal_mask seq}: T)$$
$$[\![\texttt{attend}]\!](q, k, v)[\texttt{batch}, \texttt{heads}, \texttt{seq}, \texttt{hd}] = \sum_{\texttt{seq2}} a[\texttt{batch}, \texttt{heads}, \texttt{seq}, \texttt{seq2}]\, v[\texttt{batch}, \texttt{heads}, \texttt{seq2}, \texttt{hd}]$$

3 The course GPT

`corpus/nlp/05_gpt.psk` at its course constants: a stack of six blocks as its recurrence, the attention's two contractions, dropout, the loss and the accuracy.

`05_gpt.psk`

Constants $V = 92$, $D = 384$, $H = 6$, $HD = \frac{D}{H} = 64$, $L = 6$; dimensions B, $T \leq 1024$; types `Tokens` = $\texttt{idx}[V][\texttt{batch}: B, \texttt{seq}: T]$, `Acts` = $\mathbb{R}[\texttt{batch}: B, \texttt{seq}: T, \texttt{model}: D]$, `Mask` = $\texttt{bool}[\texttt{seq}: T, \texttt{seq2}: T]$, `Acc` = (`Acts`, `key`); asserted $D \bmod H = 0$.

Module `Linear`$\{In, Out\}$. Its parameters and modules: $w : \mathbb{R}[In, Out]$, $b : \mathbb{R}[Out]$.
`Linear.apply`, of $x : \mathbb{R}[\ldots, In]$:

$$[\![\texttt{Linear.apply}]\!](\theta, x) = (\texttt{dot}\ x, w) + b$$

Module `LayerNorm`$\{Dm\}$. Its parameters and modules: $\gamma : \mathbb{R}[Dm]$, $\beta : \mathbb{R}[Dm]$.
`LayerNorm.apply`, of $x : \mathbb{R}[\ldots, \texttt{model}: Dm]$:

$$\mu = \texttt{mean axis model}\ x$$
$$\sigma_2 = \texttt{mean axis model}\ (x - \mu)^2$$
$$[\![\texttt{LayerNorm.apply}]\!](\theta, x) = \frac{x - \mu}{\sqrt{\sigma_2 + 10^{-5}}} \times \gamma + \beta$$

Module `SelfAttention`. Its parameters and modules: wq : $\texttt{Linear}\{D, D\}$, wk : $\texttt{Linear}\{D, D\}$, wv : $\texttt{Linear}\{D, D\}$, wo : $\texttt{Linear}\{D, D\}$.
`SelfAttention.apply`, of x : `Acts`, $mask$: `Mask`, kd : `key`, uses Rand:

$$
\begin{aligned}
q &= \texttt{split_heads}\ H\texttt{x}HD\ [\![\texttt{wq.apply}]\!](x) \\
k &= \texttt{split_heads}\ H\texttt{x}HD\ [\![\texttt{wk.apply}]\!](x) \\
v &= \texttt{split_heads}\ H\texttt{x}HD\ [\![\texttt{wv.apply}]\!](x) \\
s[\texttt{batch}, \texttt{heads}, \texttt{seq}, \texttt{seq2}] &= \frac{\sum_{\texttt{hd}} q[\texttt{batch}, \texttt{heads}, \texttt{seq}, \texttt{hd}]\, k[\texttt{batch}, \texttt{heads}, \texttt{seq2}, \texttt{hd}]}{\sqrt{HD}} \\
a &= \texttt{dropout}\ 0.2, kd, (\texttt{softmax axis seq2}\ s, mask) \\
u[\texttt{batch}, \texttt{heads}, \texttt{seq}, \texttt{hd}] &= \sum_{\texttt{seq2}} a[\texttt{batch}, \texttt{heads}, \texttt{seq}, \texttt{seq2}]\, v[\texttt{batch}, \texttt{heads}, \texttt{seq2}, \texttt{hd}] \\
[\![\texttt{SelfAttention.apply}]\!](\theta, x, mask, kd) &= [\![\texttt{wo.apply}]\!](\texttt{merge_heads}\ H\texttt{x}HD\ u)
\end{aligned}
$$

Module `FeedForward`. Its parameters and modules: up : $\texttt{Linear}\{D, 4{\cdot}D\}$, $down$: $\texttt{Linear}\{4{\cdot}D, D\}$.
`FeedForward.apply`, of x : `Acts`:

$$[\![\texttt{FeedForward.apply}]\!](\theta, x) = [\![\texttt{down.apply}]\!](\texttt{gelu}\ [\![\texttt{up.apply}]\!](x))$$

Module `Block`. Its parameters and modules: $attn$: `SelfAttention`, ff : `FeedForward`, ln_1 : $\texttt{LayerNorm}\{D\}$, ln_2 : $\texttt{LayerNorm}\{D\}$.
`Block.apply`, of s : `Acc`, $mask$: `Mask`, uses Rand:

$$
\begin{aligned}
(x, k) &= s \\
(katt, k_1) &= \texttt{split_key}\ 2\ k \\
(kres, k_2) &= \texttt{split_key}\ 2\ k_1 \\
(kff, k_3) &= \texttt{split_key}\ 2\ k_2 \\
h &= x + (\texttt{dropout}\ 0.2, kres, [\![\texttt{attn.apply}]\!]([\![\texttt{ln1.apply}]\!](x), mask, katt)) \\
[\![\texttt{Block.apply}]\!](\theta, s, mask) &= (h + (\texttt{dropout}\ 0.2, kff, [\![\texttt{ff.apply}]\!]([\![\texttt{ln2.apply}]\!](h))), k_3)
\end{aligned}
$$

Module `GPT`. Its parameters and modules: tok : $\mathbb{R}[\texttt{vocab}{:}\,V, \texttt{model}{:}\,D]$, pos : $\mathbb{R}[T, D]$, $blocks$: $\texttt{stack}\{L\}(\texttt{Block})$, lnf : $\texttt{LayerNorm}\{D\}$, $head$: $\mathbb{R}[D, V]$.
`GPT.apply`, of ids : `Tokens`, k : `key`, uses Rand:

$$
\begin{aligned}
mask &= \texttt{causal_mask seq:}\,T \\
h_0 &= (\texttt{embed}\ tok, ids) + (\texttt{embed}\ pos, (\texttt{positions seq:}\,T)) \\
(ke, kb) &= \texttt{split_key}\ 2\ k \\
y_0 &= ((\texttt{dropout}\ 0.2, ke, h_0), kb) \\
y_{\ell+1} &= [\![\texttt{Block.apply}]\!](\theta.blocks[\ell], y_\ell, mask) && 0 \le \ell < L \\
(hL, _) &= y_L \\
[\![\texttt{GPT.apply}]\!](\theta, ids, k) &= \texttt{dot}\ [\![\texttt{lnf.apply}]\!](hL), head
\end{aligned}
$$

Function `loss`. Of θ : `params GPT`, k : `key`, $batch$: $(\texttt{Tokens}, \texttt{Tokens})$, uses Rand:

$$
\begin{aligned}
(x, y) &= batch \\
[\![\texttt{loss}]\!](\theta, k, batch) &= \texttt{mean}\ (\texttt{cross_entropy}\ [\![\texttt{GPT.apply}]\!](\theta, x, k), y)
\end{aligned}
$$

Function `hits`. Of p : $\mathbb{R}[\texttt{batch}{:}\,B, \texttt{seq}{:}\,T, \texttt{vocab}{:}\,V]$, q : $\mathbb{R}[\texttt{batch}{:}\,B, \texttt{seq}{:}\,T, \texttt{vocab}{:}\,V]$:

$$[\![\texttt{hits}]\!](p, q) = \texttt{sum axis vocab}\ (p \times q)$$

Function `accuracy`. Of θ : `params GPT`, k : `key`, $batch$: (`Tokens`, `Tokens`), uses Rand:

$$(x, y) = batch$$
$$[\![\texttt{accuracy}]\!](\theta, k, batch) = \texttt{mean}\ [\![\texttt{hits}]\!]((\texttt{one_hot}\ (\texttt{argmax}\ [\![\texttt{GPT.apply}]\!](\theta, x, k))), (\texttt{one_hot}\ y))$$

4 The adjoints of grad

PSK-Core after differentiation of `tutorial/04_grad.psk`, one equation per binding (definition 3.39), each binding of a generated function with what it was made for: the forward pass's copy of a binding, or a contribution to an adjoint by the rule of its primitive, as table `core-adjoint` writes it.

Function `Linear.apply`. Of θ : `params Linear{In, Out}`, $x : \mathbb{R}[\ldots, In]$, dimensions In, Out.

$$w = \theta.w$$
$$\%3 = \texttt{dot}\ x, w$$
$$b = \theta.b$$
$$\%5 = \%3 + b$$
$$[\![\texttt{Linear.apply}]\!](\theta, x) = \%5$$

Function `loss`. Of θ : `params Linear{D, 1}`, $x : \mathbb{R}[\texttt{batch}: B, D]$, $y : \mathbb{R}[\texttt{batch}: B, 1]$, dimensions D, B.

$$\hat{y} = [\![\texttt{Linear.apply}\{In = D, Out = 1\}]\!](\theta, x)$$
$$\%4 = \hat{y} - y$$
$$\%5 = \%4^2$$
$$\%6 = \texttt{mean}\ \%5$$
$$[\![\texttt{loss}]\!](\theta, x, y) = \%6$$

Function `loss_grad`. Of θ : `params Linear{D, 1}`, $x : \mathbb{R}[\texttt{batch}: B, D]$, $y : \mathbb{R}[\texttt{batch}: B, 1]$, dimensions D, B.

$$\%3 = v \mapsto [\![\texttt{loss_grad.lambda0}]\!](x, y, v)$$
$$\%4 = v \mapsto [\![\texttt{loss_grad.lambda0.grad}]\!](x, y, v)$$
$$\%5 = \%4(\theta)$$
$$[\![\texttt{loss_grad}]\!](\theta, x, y) = \%5$$

Function `Linear.init`. Of key : `key`, dimensions In, Out, uses Rand.

$$\%1 = \texttt{split_key}\ 2\ key$$
$$\%2 = \%1.0$$
$$\%3 = \%1.1$$
$$w = \texttt{xavier_uniform}\ \%2$$
$$b = \texttt{zeros}$$
$$\%6 = \{w = w, b = b\}$$
$$[\![\texttt{Linear.init}]\!](key) = \%6$$

Function `loss_grad.lambda0`. Of $x : \mathbb{R}[\texttt{batch}: B, D]$, $y : \mathbb{R}[\texttt{batch}: B, 1]$, θ : `params Linear{D, 1}`, dimensions D, B.

$$\%3 = [\![\texttt{loss}\{D = D, B = B\}]\!](\theta, x, y)$$
$$[\![\texttt{loss_grad.lambda0}]\!](x, y, \theta) = \%3$$

Function `loss_grad.lambda0.vjp`. Generated by differentiation (`vjp`) from `loss_grad.lambda0`. Of $x : \mathbb{R}$[`batch`: B, D], $y : \mathbb{R}$[`batch`: $B, 1$], θ : `params Linear{D, 1}`, $ct : \mathbb{R}[]$, dimensions D, B.

$$\begin{aligned}
\%4 &= [\![\texttt{loss_grad.lambda0.fwd}\{D = D, B = B\}]\!](x, y, \theta) \\
\%5 &= \%4.0 \\
\%6 &= [\![\texttt{loss_grad.lambda0.bwd}\{D = D, B = B\}]\!](x, \theta, \%5, ct) \\
[\![\texttt{loss_grad.lambda0.vjp}]\!](x, y, \theta, ct) &= \%6
\end{aligned}$$

Function `loss.vjp`. Generated by differentiation (`vjp`) from `loss`. Of θ : `params Linear{D, 1}`, $x : \mathbb{R}$[`batch`: B, D], $y : \mathbb{R}$[`batch`: $B, 1$], $ct : \mathbb{R}[]$, dimensions D, B.

$$\begin{aligned}
\%4 &= [\![\texttt{loss.fwd}\{D = D, B = B\}]\!](\theta, x, y) \\
\%5 &= \%4.0 \\
\%6 &= [\![\texttt{loss.bwd}\{D = D, B = B\}]\!](\theta, x, \%5, ct) \\
[\![\texttt{loss.vjp}]\!](\theta, x, y, ct) &= \%6
\end{aligned}$$

Function `Linear.apply.vjp`. Generated by differentiation (`vjp`) from `Linear.apply`. Of θ : `params Linear{In, Out}`, $x : \mathbb{R}[\ldots, In]$, $ct : \mathbb{R}[\ldots, Out]$, dimensions In, Out.

$$\begin{aligned}
\%3 &= [\![\texttt{Linear.apply.bwd}\{In = In, Out = Out\}]\!](\theta, x, ct) \\
[\![\texttt{Linear.apply.vjp}]\!](\theta, x, ct) &= \%3
\end{aligned}$$

Function `loss_grad.lambda0.grad`. Generated by differentiation (`grad`) from `loss_grad.lambda0`. Of $x : \mathbb{R}$[`batch`: B, D], $y : \mathbb{R}$[`batch`: $B, 1$], θ : `params Linear{D, 1}`, dimensions D, B.

$$\begin{aligned}
\%3 &= 1 && \text{the cotangent 1} \\
\%4 &= \%3 && \text{the cotangent 1} \\
\%5 &= [\![\texttt{loss_grad.lambda0.vjp}\{D = D, B = B\}]\!](x, y, \theta, \%4) \\
\%6 &= \%5.0 \\
[\![\texttt{loss_grad.lambda0.grad}]\!](x, y, \theta) &= \%6
\end{aligned}$$

Function `Linear.apply.bwd`. Generated by differentiation (`bwd`) from `Linear.apply`. Of θ : `params Linear{In, Out}`, $x : \mathbb{R}[\ldots, In]$, $ct : \mathbb{R}[\ldots, Out]$, dimensions In, Out.

$$\begin{aligned}
b &= \theta.b && \text{primal } b\text{, kept} \\
\%8 &= \texttt{sum_to}\ ct, b && \text{adjoint of \%5 (}\texttt{add}\text{)} \\
\%9 &= \texttt{zeros_like}\ \theta && \text{adjoint of } b \text{ (}\texttt{field}\text{)} \\
\%10 &= \texttt{set_field b}\ \%9, \%8 && \text{adjoint of } b \text{ (}\texttt{field}\text{)} \\
\%12 &= \texttt{dot_vjp_r}\ x, ct && \text{adjoint of \%3 (}\texttt{dot}\text{)} \\
\%13 &= \texttt{zeros_like}\ \theta && \text{adjoint of } w \text{ (}\texttt{field}\text{)} \\
\%14 &= \texttt{set_field w}\ \%13, \%12 && \text{adjoint of } w \text{ (}\texttt{field}\text{)} \\
\%15 &= \%10 + \%14 && \text{accumulates the adjoint of a parameter of } \texttt{Linear.apply} \\
\%16 &= (\%15) \\
[\![\texttt{Linear.apply.bwd}]\!](\theta, x, ct) &= \%16
\end{aligned}$$

Function `loss.bwd`. Generated by differentiation (`bwd`) from `loss`. Of θ : `params Linear{D, 1}`, $x : \mathbb{R}[\texttt{batch}: B, D]$, $residuals : (\mathbb{R}[\texttt{batch}: B, 1], \mathbb{R}[\texttt{batch}: B, 1])$, $ct : \mathbb{R}[]$, dimensions D, B.

$$
\begin{aligned}
\%5 &= residuals.0 && \text{primal \%4, kept}\\
\%6 &= residuals.1 && \text{primal \%5, kept}\\
\%8 &= \texttt{mean_vjp}\ ct, \%6 && \text{adjoint of \%6 (}\texttt{mean}\text{)}\\
\%9 &= 2 && \text{adjoint of \%5 (}\texttt{pow}\text{)}\\
\%10 &= \%9 && \text{adjoint of \%5 (}\texttt{pow}\text{)}\\
\%11 &= \%5^{1} && \text{adjoint of \%5 (}\texttt{pow}\text{)}\\
\%12 &= \%10 \times \%11 && \text{adjoint of \%5 (}\texttt{pow}\text{)}\\
\%13 &= \%8 \times \%12 && \text{adjoint of \%5 (}\texttt{pow}\text{)}\\
\%17 &= [\![\texttt{Linear.apply.bwd}\{In = D, Out = 1\}]\!](\theta, x, \%13) && \text{adjoint of } \hat{y} \text{ (}\texttt{call}\text{)}\\
\%18 &= \%17.0 && \text{adjoint of } \hat{y} \text{ (}\texttt{call}\text{)}\\
\%20 &= (\%18)\\
[\![\texttt{loss.bwd}]\!](\theta, x, residuals, ct) &= \%20
\end{aligned}
$$

Function `loss.fwd`. Generated by differentiation (`fwd`) from `loss`. Of θ : `params Linear{D, 1}`, $x : \mathbb{R}[\texttt{batch}: B, D]$, $y : \mathbb{R}[\texttt{batch}: B, 1]$, dimensions D, B.

$$
\begin{aligned}
\hat{y} &= [\![\texttt{Linear.apply}\{In = D, Out = 1\}]\!](\theta, x) && \text{primal } \hat{y}\text{, stored}\\
\%4 &= \hat{y} - y && \text{primal \%4, stored}\\
\%5 &= \%4^{2} && \text{primal \%5, stored}\\
\%7 &= (\%4, \%5)\\
\%8 &= (\%7)\\
[\![\texttt{loss.fwd}]\!](\theta, x, y) &= \%8
\end{aligned}
$$

Function `loss_grad.lambda0.bwd`. Generated by differentiation (`bwd`) from `loss_grad.lambda0`. Of $x : \mathbb{R}[\texttt{batch}: B, D]$, θ : `params Linear{D, 1}`, $residuals : ((\mathbb{R}[\texttt{batch}: B, 1], \mathbb{R}[\texttt{batch}: B, 1]))$, $ct : \mathbb{R}[]$, dimensions D, B.

$$
\begin{aligned}
\%10 &= residuals.0\\
\%5 &= [\![\texttt{loss.bwd}\{D = D, B = B\}]\!](\theta, x, \%10, ct) && \text{adjoint of \%3 (}\texttt{call}\\
\%6 &= \%5.0 && \text{adjoint of \%3 (}\texttt{call}\\
\%9 &= (\%6)\\
[\![\texttt{loss_grad.lambda0.bwd}]\!](x, \theta, residuals, ct) &= \%9
\end{aligned}
$$

Function `loss_grad.lambda0.fwd`. Generated by differentiation (`fwd`) from `loss_grad.lambda0`. Of $x : \mathbb{R}[\texttt{batch}: B, D]$, $y : \mathbb{R}[\texttt{batch}: B, 1]$, θ : `params Linear{D, 1}`, dimensions D, B.

$$
\begin{aligned}
\%4 &= [\![\texttt{loss.fwd}\{D = D, B = B\}]\!](\theta, x, y)\\
\%5 &= \%4.0\\
\%6 &= (\%5)\\
\%7 &= (\%6)\\
[\![\texttt{loss_grad.lambda0.fwd}]\!](x, y, \theta) &= \%7
\end{aligned}
$$